

Bulid Array from premutation :

Given a **zero-based permutation** `nums` (**0-indexed**), build an array `ans` of the **same length** where `ans[i] = nums[nums[i]]` for each $0 \leq i < \text{nums.length}$ and return it.

A **zero-based permutation** `nums` is an array of **distinct** integers from 0 to `nums.length - 1` (**inclusive**).

Example 1:

Input: `nums = [0,2,1,5,3,4]`

Output: `[0,1,2,4,5,3]`

Explanation: The array `ans` is built as follows:

```
ans = [nums[nums[0]], nums[nums[1]], nums[nums[2]], nums[nums[3]],
nums[nums[4]], nums[nums[5]]]

= [nums[0], nums[2], nums[1], nums[5], nums[3], nums[4]]

= [0,1,2,4,5,3]
```

Example 2:

Input: `nums = [5,0,1,2,3,4]`

Output: `[4,5,0,1,2,3]`

Explanation: The array `ans` is built as follows:

```
ans = [nums[nums[0]], nums[nums[1]], nums[nums[2]], nums[nums[3]],
nums[nums[4]], nums[nums[5]]]

= [nums[5], nums[0], nums[1], nums[2], nums[3], nums[4]]

= [4,5,0,1,2,3]
```

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $0 \leq \text{nums}[i] < \text{nums.length}$
- The elements in `nums` are **distinct**.

Program :

```
class Solution
public int[] buildArray(int[] nums) {
    int[] ans = new int[nums.length];

    for (int i = 0; i < nums.length; i++) {
        ans[i] = nums[nums[i]];
    }
    return ans;
}
```

Output :

The screenshot shows a web-based code execution environment. At the top, there are two tabs: 'Testcase' (selected) and 'Test Result'. Below the tabs, the word 'Accepted' is displayed in green, followed by 'Runtime: 0 ms'. Underneath, there are two buttons: 'Case 1' and 'Case 2', both with green checkmarks. The 'Input' section shows 'nums =' followed by the array '[0,2,1,5,3,4]'. The 'Output' section shows the array '[0,1,2,4,5,3]'. The 'Expected' section also shows the array '[0,1,2,4,5,3]'. A vertical scrollbar is visible on the left side of the interface.

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =
[0,2,1,5,3,4]

Output

[0,1,2,4,5,3]

Expected

[0,1,2,4,5,3]

Shuffle the Array :

Given the array nums consisting of $2n$ elements in the form $[x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n]$.

Return the array in the form $[x_1, y_1, x_2, y_2, \dots, x_n, y_n]$.

Example 1:

Input: nums = [2,5,1,3,4,7], n = 3

Output: [2,3,5,4,1,7]

Explanation: Since $x_1=2$, $x_2=5$, $x_3=1$, $y_1=3$, $y_2=4$, $y_3=7$ then the answer is [2,3,5,4,1,7].

Example 2:

Input: nums = [1,2,3,4,4,3,2,1], n = 4

Output: [1,4,2,3,3,2,4,1]

Example 3:

Input: nums = [1,1,2,2], n = 2

Output: [1,2,1,2]

Constraints:

- $1 \leq n \leq 500$
- `nums.length == 2n`
- $1 \leq \text{nums}[i] \leq 10^3$

Program :

```
class Solution {  
    public int[] shuffle(int[] nums, int n) {  
        int[] ans = new int[2 * n];  
  
        for (int i = 0; i < n; i++) {  
            ans[2 * i] = nums[i];    // x  
            ans[2 * i + 1] = nums[i + n]; // y  
        }  
    }  
}
```

```
    }  
  
    return ans;  
}  
}
```

Output :

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

✓ Case 3

Input
nums =
[2,5,1,3,4,7]

n =
3

Output
[2,3,5,4,1,7]

Expected

Given an integer array `nums` and an integer `val`, remove all occurrences of `val` in `nums` [in-place](#). The order of the elements may be changed. Then return *the number of elements in `nums` which are not equal to `val`*.

Consider the number of elements in `nums` which are not equal to `val` be `k`, to get accepted, you need to do the following things:

- Change the array `nums` such that the first `k` elements of `nums` contain the elements which are not equal to `val`. The remaining elements of `nums` are not important as well as the size of `nums`.
- Return `k`.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int val = ...; // Value to remove
int[] expectedNums = [...]; // The expected answer with correct length.
    // It is sorted with no values equaling val.
```

```
int k = removeElement(nums, val); // Calls your implementation
```

```
assert k == expectedNums.length;
sort(nums, 0, k); // Sort the first k elements of nums
for (int i = 0; i < actualLength; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be accepted.

Example 1:

Input: `nums = [3,2,2,3]`, `val = 3`

Output: 2, `nums = [2,2,_,_]`

Explanation: Your function should return `k = 2`, with the first two elements of `nums` being 2.

It does not matter what you leave beyond the returned k (hence they are underscores).

Example 2:

Input: nums = [0,1,2,2,3,0,4,2], val = 2

Output: 5, nums = [0,1,4,0,3,_,_,_]

Explanation: Your function should return k = 5, with the first five elements of nums containing 0, 0, 1, 3, and 4.

Note that the five elements can be returned in any order.

It does not matter what you leave beyond the returned k (hence they are underscores).

Constraints:

- $0 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 50$
- $0 \leq \text{val} \leq 100$

Program :

```
class Solution {
    public int removeElement(int[] nums, int val) {
        int k = 0;

        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != val) {
                nums[k] = nums[i];
                k++;
            }
        }

        return k;
    }
}
```

Output :

☒ Testcase ☒ Test Result

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

nums =
[3,2,2,3]

val =
3

Output

[2,2]

Remove duplicates from sorted array :

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**.

Consider the number of *unique elements* in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in **sorted order**. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
```

```
int[] expectedNums = [...]; // The expected answer with correct length
```

```
int k = removeDuplicates(nums); // Calls your implementation
```

```
assert k == expectedNums.length;
```

```
for (int i = 0; i < k; i++) {
```

```
    assert nums[i] == expectedNums[i];
```

```
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

Input: nums = [1,1,2]

Output: 2, nums = [1,2,_]

Explanation: Your function should return k = 2, with the first two elements of nums being 1 and 2 respectively.

It does not matter what you leave beyond the returned k (hence they are underscores).

Example 2:

Input: nums = [0,0,1,1,1,2,2,3,3,4]

Output: 5, nums = [0,1,2,3,4,_,_,_,_,_]

Explanation: Your function should return k = 5, with the first five elements of nums being 0, 1, 2, 3, and 4 respectively.

It does not matter what you leave beyond the returned k (hence they are underscores).

Constraints:

- $1 \leq \text{nums.length} \leq 3 \times 10^4$
- $-100 \leq \text{nums}[i] \leq 100$
- nums is sorted in **non-decreasing** order.

Program :

```
class Solution {
```

```
    public int removeDuplicates(int[] nums) {
```

```
int k = 1;

for (int i = 1; i < nums.length; i++) {
    if (nums[i] != nums[i - 1]) {
        nums[k] = nums[i];
        k++;
    }
}

return k;
}
```

☒ Testcase | [> Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

Input

nums =
[1,1,2]

Output

[1,2]

Expected

[1,2]



Maximum Subarray :

Given an integer array `nums`, find the subarray with the largest sum, and return *its sum*.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`

Output: 6

Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`

Output: 1

Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5,4,-1,7,8]`

Output: 23

Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

Program :

```
class Solution {  
    public int maxSubArray(int[] nums) {  
        int currentSum = nums[0];  
        int maxSum = nums[0];  
  
        for (int i = 1; i < nums.length; i++) {
```

```
        currentSum = Math.max(nums[i], currentSum + nums[i]);  
        maxSum = Math.max(maxSum, currentSum);  
    }  
  
    return maxSum;  
}  
}
```

Output :

Testcase | >_ Test Result

Accepted Runtime: 0 ms

✓ Case 1 ✓ Case 2 ✓ Case 3

Input

nums =
[-2, 1, -3, 4, -1, 2, 1, -5, 4]

Output

6

Expected

6

Find the highest altitude :

There is a biker going on a road trip. The road trip consists of $n + 1$ points at various altitudes. The biker starts his trip on point 0 with altitude equal 0.

You are given an integer array `gain` of length n where `gain[i]` is the **net gain in altitude** between points i and $i + 1$ for all $(0 \leq i < n)$. Return *the **highest altitude** of a point*.

Example 1:

Input: `gain = [-5,1,5,0,-7]`

Output: 1

Explanation: The altitudes are `[0,-5,-4,1,1,-6]`. The highest is 1.

Example 2:

Input: `gain = [-4,-3,-2,-1,4,3,2]`

Output: 0

Explanation: The altitudes are `[0,-4,-7,-9,-10,-6,-3,-1]`. The highest is 0.

Constraints:

- $n == \text{gain.length}$
- $1 \leq n \leq 100$
- $-100 \leq \text{gain}[i] \leq 100$

Program :

```
class Solution {  
    public int largestAltitude(int[] gain) {  
        int altitude = 0;  
        int maxAltitude = 0;  
  
        for (int i = 0; i < gain.length; i++) {
```

```
    altitude += gain[i];

    if (altitude > maxAltitude) {
        maxAltitude = altitude;
    }
}

return maxAltitude;
}
```

Output :

✓ Testcase | >_ Test Result

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

gain =
[-5, 1, 5, 0, -7]

Output

1

Expected

1

Group Anagrams :

Given an array of strings `strs`, group the anagrams together. You can return the answer in any order.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Explanation:

- There is no string in `strs` that can be rearranged to form "bat".
- The strings "nat" and "tan" are anagrams as they can be rearranged to form each other.
- The strings "ate", "eat", and "tea" are anagrams as they can be rearranged to form each other.

Example 2:

Input: `strs = [""]`

Output: `[[""]]`

Example 3:

Input: `strs = ["a"]`

Output: `[["a"]]`

Constraints:

- $1 \leq \text{strs.length} \leq 10^4$
- $0 \leq \text{strs}[i].\text{length} \leq 100$
- `strs[i]` consists of lowercase English letters.

Program :

```
import java.util.*;
```

```
class Solution {
```

```

public List<List<String>> groupAnagrams(String[] strs)

Map<String, List<String>> map = new HashMap<>();

for (String s : strs) {
    char[] chars = s.toCharArray();
    Arrays.sort(chars);

    String key = new String(chars);

    if (!map.containsKey(key)) {
        map.put(key, new ArrayList<>());
    }

    map.get(key).add(s);
}

return new ArrayList<>(map.values());
}
}

```

Output :

☒ Testcase
 ☒ Test Result

Accepted
 Runtime: 0 ms

☒ Case 1
 ☒ Case 2
 ☒ Case 3

Input


```

strs =
["eat","tea","tan","ate","nat","bat"]
    
```

Output


```

[["eat","tea","ate"],["bat"],["tan","nat"]]
    
```

Expected


```

[["bat"],["nat","tan"],["ate","eat","tea"]]
    
```

Top K Frequent elements :

Given an integer array `nums` and an integer `k`, return *the k most frequent elements*. You may return the answer in any order.

Example 1:

Input: `nums = [1,1,1,2,2,3]`, `k = 2`

Output: `[1,2]`

Example 2:

Input: `nums = [1]`, `k = 1`

Output: `[1]`

Example 3:

Input: `nums = [1,2,1,2,1,2,3,1,3,2]`, `k = 2`

Output: `[1,2]`

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `k` is in the range `[1, the number of unique elements in the array]`.
- It is guaranteed that the answer is unique.

Program :

```
import java.util.*;
```

```
class Solution {
```

```
    public int[] topKFrequent(int[] nums, int k) {
```

```
        // Count frequency of each number
```

```
        Map<Integer, Integer> map = new HashMap<>();
```

```
        for (int num : nums) {
```

```

        map.put(num, map.getOrDefault(num, 0) + 1);
    }

    // Min-heap based on frequency
    PriorityQueue<Integer> pq =
        new PriorityQueue<>((a, b) -> map.get(a) - map.get(b));

    for (int num : map.keySet()) {
        pq.add(num);

        if (pq.size() > k) {
            pq.poll();
        }
    }

    // Store result
    int[] result = new int[k];

    for (int i = k - 1; i >= 0; i--) {
        result[i] = pq.poll();
    }

    return result;
}

```

Output :

☒ Testcase | [> Test Result](#)

Accepted Runtime: 2 ms

☒ Case 1

☒ Case 2

☒ Case 3

Input

```
nums =  
[1,1,1,2,2,3]
```

```
k =  
2
```

Output

```
[1,2]
```

In computer science, a double-ended queue (dequeue, often abbreviated to deque, pronounced deck) is an abstract data type that generalizes a queue, for which elements can be added to or removed from either the front (head) or back (tail).

Deque interfaces can be implemented using various types of collections such as `LinkedList` or `ArrayDeque` classes. For example, deque can be declared as:

```
Deque deque = new LinkedList<>();
```

or

```
Deque deque = new ArrayDeque<>();
```

You can find more details about Deque [here](#).

In this problem, you are given `n` integers. You need to find the maximum number of unique integers among all the possible contiguous subarrays of size `k`.

Note: Time limit is `2` second for this problem.

Input Format

The first line of input contains two integers `n` and `k`: representing the total number of integers and the size of the subarray, respectively. The next line contains `n` space separated integers.

Constraints

The numbers in the array will range between `1` to `10^5`.

Output Format

Print the *maximum* number of unique integers among all possible contiguous subarrays of size `k`.

Sample Input

```
6 3
```

```
5 3 5 2 3 2
```

Sample Output

```
3
```

Explanation

In the sample testcase, there are 4 subarrays of contiguous numbers.

- Has unique numbers.
- Has unique numbers.
- Has unique numbers.
- Has unique numbers.

In these subarrays, there are unique numbers, respectively. The maximum amount of unique numbers among all possible contiguous subarrays is .

Program :

```
import java.util.*;

public class Solution {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int m = sc.nextInt();

        int[] nums = new int[n];

        for (int i = 0; i < n; i++) {
            nums[i] = sc.nextInt();
        }

        Map<Integer, Integer> map = new HashMap<>();

        int maxUnique = 0;

        // First window
```

```
for (int i = 0; i < m; i++) {  
    map.put(nums[i], map.getDefault(nums[i], 0) + 1);  
}  
  
maxUnique = map.size();  
  
// Slide the window  
for (int i = m; i < n; i++) {  
  
    // Add new element  
    map.put(nums[i], map.getDefault(nums[i], 0) + 1);  
  
    // Remove old element  
    int old = nums[i - m];  
  
    map.put(old, map.get(old) - 1);  
  
    if (map.get(old) == 0) {  
        map.remove(old);  
    }  
  
    // Update maximum  
    maxUnique = Math.max(maxUnique, map.size());  
}  
  
System.out.println(maxUnique);  
}  
}
```

Output :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

Input (stdin)

[Download](#)

```
1 6 3
2 5 3 5 2 3 2
```

Your Output (stdout)

```
1 3
```

Expected Output

[Download](#)

```
1 3
```

In computer science, a set is an abstract data type that can store certain values, without any particular order, and no repeated values(Wikipedia). is an example of a set, but is not a set. Today you will learn how to use sets in java by solving this problem.

You are given pairs of strings. Two pairs and are identical if and . That also implies is *not* same as . After taking each pair as input, you need to print number of unique pairs you currently have.

Complete the code in the editor to solve this problem.

Input Format

In the first line, there will be an integer denoting number of pairs. Each of the next lines will contain two strings separated by a single space.

Constraints:

-
- Length of each string is atmost and will consist lower case letters only.

Output Format

Print lines. In the line, print number of unique pairs you have after taking pair as input.

Sample Input

```
5
john tom
john mary
john tom
mary anna
mary anna
```

Sample Output

```
1
2
2
3
3
```

Explanation

- After taking the first input, you have only one pair: (john,tom)
- After taking the second input, you have two pairs: (john, tom) and (john, mary)
- After taking the third input, you still have two unique pairs.
- After taking the fourth input, you have three unique pairs: (john,tom), (john, mary) and (mary, anna)
- After taking the fifth input, you still have three unique pairs.

Program :

```
import java.util.*;

public class Solution {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        HashSet<String> set = new HashSet<>();

        for (int i = 0; i < n; i++) {

            String first = sc.next();

            String second = sc.next();

            String pair = first + " " + second;

            set.add(pair);

            System.out.println(set.size());
```

```
}
```

```
sc.close();
```

```
}
```

```
}
```